Autonomous Driving Simulation with Reinforcement Learning

Michelangelo Zampieri

CS 5100 Foundations of Artificial Intelligence

Spring 2026

Introduction

The aim of this project was to create a simplified autonomous driving car environment and train a simple reinforcement learning agent on said environment. The concept for this project came from the various discussions in class regarding the use of agents and AI techniques for such a task, and although the real-world task is out of the scope for this project, a simplified digital environment project is feasible. This project was further developed with *Deep Reinforcement Learning for Autonomous Driving: A Survey*, by Kiran *et al.*, as a basis. This paper aimed for a more involved but similar simulation of the task, of creating a self driving agent with reinforcement learning, and discusses various approaches at how one can approach training the self driving agent. This paper details the theory behind and various components of self-driving vehicles, and discusses various techniques to train agents for these tasks. For the purposes of this project, however, the topic of Q-learning is the most relevant. Out of the various approaches discussed in the paper, this method was chosen for this project because it is considered to be the most straightforward approach. Q-learning also guarantees convergence to the optimal value at each state, given that there is sufficient training data.

Methods

The Environment

As mentioned in the introduction, the method implemented to train the agent is the Q-learning algorithm. However, training the agent is only half of the project. The first half was creating an interactive environment for the agent to learn and interact in, and the environment that was created using several classes that interact together.

First is the Cell class This class defines a cell in the grid that forms the map, this cell will hold various objects that are described in the following paragraphs. Second is the Car class,

which defines a car obstacle that the agent will have to try to avoid. Third is the Highway class, this class holds an internal 3 x 3 grid with cars randomly placed in them. Highways are also directional, they can be vertical or horizontal, and the agent can only traverse them in the assigned direction. Additionally, highways are guaranteed to be passable in the defined direction, this means that cars inside will never be placed such that the agent can never traverse the highway cell, this was done in order to avoid the edge case of an agent not being able to traverse a given highway cell due to both sides being blocked by cars. These classes are brought together in the World class, which creates the grid of Cells and populates the map. The map is filled by first creating one path from start (always top left) to goal (always bottom right) with highways, this ensures that the agent can always solve the maze. After the main path is created, other branching paths are added from the main path at random. The path's parameters can be changed by the user via the `num_branches` and `max_length` parameters. Finally, cars are placed randomly in non-highway cells. This generation ensures that the agent can always reach the goal either by highways or by roads (non highway cells).

The aim of this environment is to determine how the agent chooses to navigate highways versus roads, as the highways will get them there faster, higher reward, but contain more cars, and more movement within the cell, higher potential for crashing. The agent could alternatively choose to take non highway cells which have less cars but lower reward, or a mix of both highways and roads. Finally the whole environment is wrapped with the Gymnasium package methods in the Car_Env class, this makes interacting with the environment and training the agent much easier than doing it without the Gymnasium package. This class and the methods defined will be the main entry point of the agent.

Additionally, the project also features a GUI created in World_Visualizer via the Matplotlib package. This GUI shows how the agent navigates through the environment and shows all obstacles, highways and roads. Users can also click on the highway cells to see the locations of cars within them.

The Agent

The agent is defined in the Agent class. It is initialized with an environment instance and values for the various training variables, as well as an empty Q table. In this class, several methods are defined, used in the Q-learning algorithm: `_q` returns the q-value for a state, action pair, ` _best_action` returns the action with the highest Q-value for the given state, `_update` updates the Q-table based on the observed transition and rewards, `choose_action` uses the epsilon greedy strategy to choose whether to take the optimal or a random move, and `decay_epsilon` reduces epsilon after each training episode. Finally, `run_episode` combines all of the previous methods to run an episode of training. First, the agent extracts the information from the environment with the reset method from Car_Env, and initializes rewards and steps counts. Then, while the episode is not terminated, the next action is chosen with an epsilon greedy strategy and the action is taken. Information from the new environment is extracted, the Q-table is updated, rewards and steps are increased, and epsilon is decayed. This is one full episode of training, and when completed the rewards and steps are returned. These will be used in the evaluation, to gather various information and statistics on the agent and the training.

The Agent class also has a save and load method that saves and loads the Q-table as a Pickle file in the project directory, making training and evaluation simpler.

Training

The training of the model is done in the file train.py. This script starts by defining paths of where the agent file is stored as well as defining episodes and other parameters. Next, two world environments are created, the first is fixed, and the parameters can be set by the user, and the second is a randomized world. Training is split into two phases, in the first phase the agent continuously trains on the same environment, this allows the agent to learn general strategies and teaches it how to navigate and interact with the environment. In the second phase the agent is put into the random environment, which will be different from the first, which will teach the agent to adapt to uncertainties and unseen situations. Both phases of training are executed in the same way, with the only difference being the environment given. The training loop for the specified number of iterations and records the rewards and steps taken. The loop also keeps track of the rolling rewards, which are used for evaluation. Finally, the script prints the average rewards and steps every few episodes. At the end of training, the results are saved in a Numpy array and a training curve for steps and rewards is created and saved. The last thing the agent does in the training script is run a final, completely greedy run of the phase one environment, allowing the user to see if the optimal solution found by training has learned optimal strategies that lead to improvements in the fixed environment.

Rewards

Rewards are an important part of any reinforcement learning algorithm, and can have significant impact on the performance of a model. When formulating the concept for the project, the main metrics that were evaluated were safety and efficiency. These metrics are reflected in the rewards and the reward distributions.

Reaching Goal	+ 200	Reaching the goal is the ultimate goal
Highway Step	+ 5	Using a highway is efficient
Switching Lane in Highway	+ 4	Avoiding a car and making progress
Non Highway Step	- 3	Slower method to get to goal
Invalid Step	- 1	Slight penalty if an invalid move is made
Hitting a Car	- 40	Big penalty for hitting a car

It is important to note that with the current reward distribution, a viable strategy would be to just enter and exit a highway cell indefinitely. To avoid this, there is a check in the training pipeline to increasingly penalize any agent stuck in such a loop.

Hyperparameters

Hyperparameters and their tuning are an essential part of this project. There are many hyperparameters used in training the agent, and figuring out the optimal value for them is essential for creating the best possible agent. This is done in the script optimize.py. In this script, several key hyperparameters are placed in a grid search in order to optimize their values. The following hyperparameters are tested: learning rate (alpha), gamma, epsilon, phase_one_episodes, and phase_two_episodes. The following values for each parameter are tested:

alpha	0.01	0.05	0.1	0.3	0.5
gamma	0.8	0.9	0.95	0.99	0.999
epsilon_decay	0.990	0.995	0.998	0.999	

phase1_episodes	500	1500	3000	5000	
phase2_episodes	500	1000	2000	5000	

This grid search creates a total of 1,600 agents to train. The optimize.py script also returns a list of the top ten best agents which can be used for further exploration.

Evaluation

Evaluation of the model was done in the script evaluation.py. This script loads an agent from a saved pickle file and runs the agent on a shorter number of episodes of a fixed and then random environment. This evaluation loop is similar to the training loop, but the Q-table is not updated and various statistics are extracted and calculated. These statistics are used to generate figures after the evaluation is complete. To make the evaluation more insightful, a wholly random agent is also trained simultaneously on another copy of the environment, giving a baseline with which the trained agent can be compared. The evaluation script calculates the following metrics and reports them in the summary graphs: Goal completion, rewards, efficiency, highway usage, and safety.

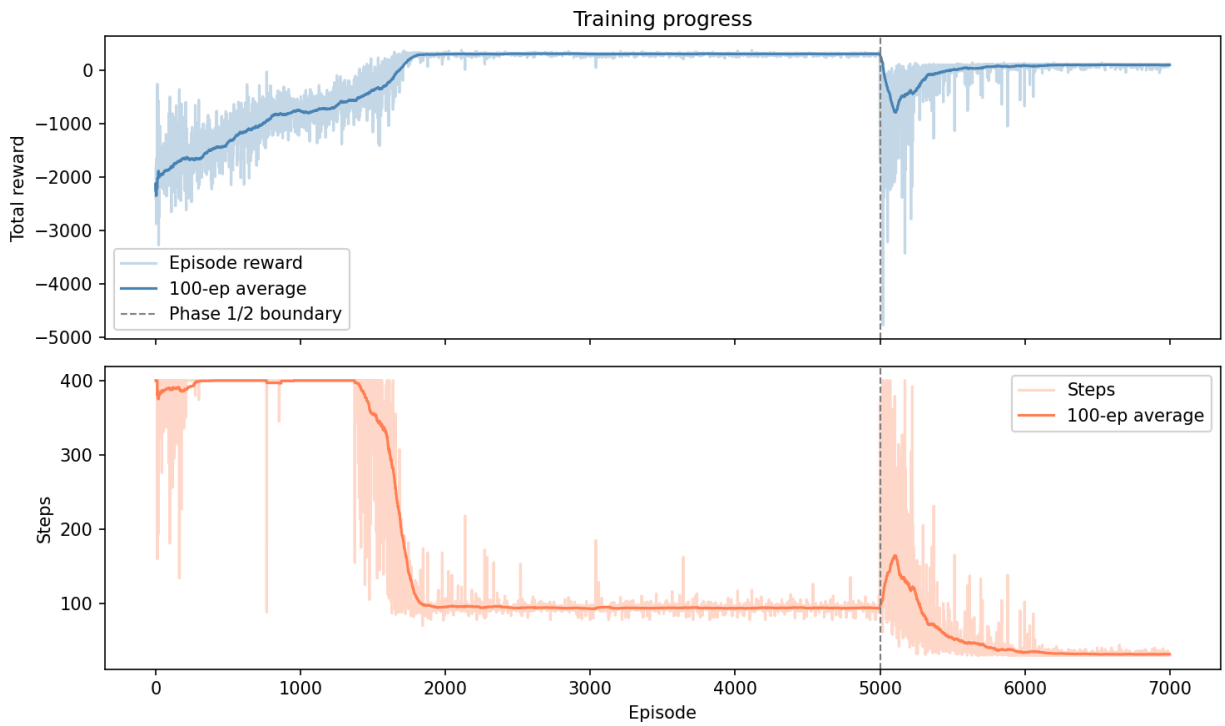
Results

Initial Results

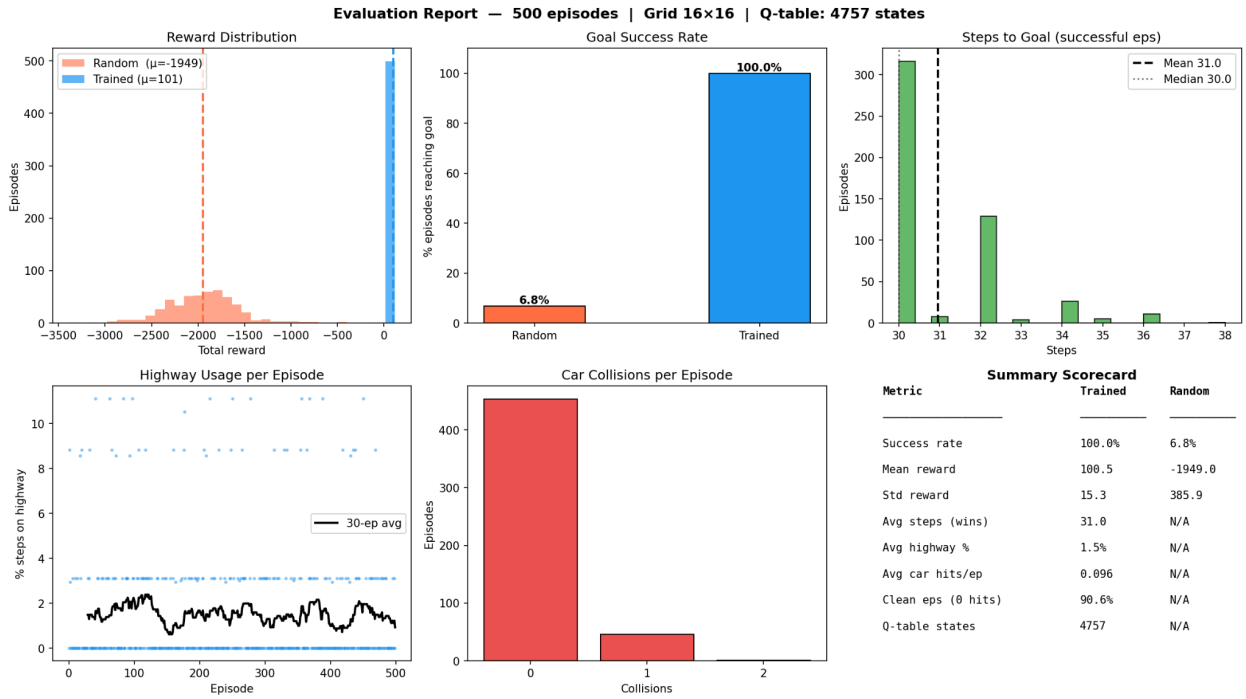
In order to get a better sense of the performance of the agent, three agents were trained. The first is a random agent, which just takes random movements. This agent acts as a worst case scenario for the model, but is a useful baseline for comparison. The second agent is an initial agent, which uses the initial hyperparameters without hyperparameter optimizations. This is another baseline agent, and is used as reference to determine if the hyperparameter optimization was successful in optimizing agent performance. The last agent uses the optimal hyper

parameters found from the grid search. This agent will theoretically perform the best. All trained agents were trained and evaluated on environments with the same parameters, but due to the randomness of the generation they unavoidably differ.

The initial agent was trained with the following hyperparameters: alpha = 0.1, gamma = 0.99, epsilon_decay = 0.998, phase1_episodes = 5000, and phase2_episodes = 2000. This agent had a decent performance with training and has the following training curves:



The initial agent shows a final average reward of 55 and average step count of 35. Considering that the agent was trained on a 16 x 16 grid, it seems to have found an efficient way of traversing the world. Additionally, a reward of 55 indicates that the agent was able to reach the goal, but lost a significant amount points on the way, likely due to collisions. The evaluate.py script reveals more information on the path the agent took:

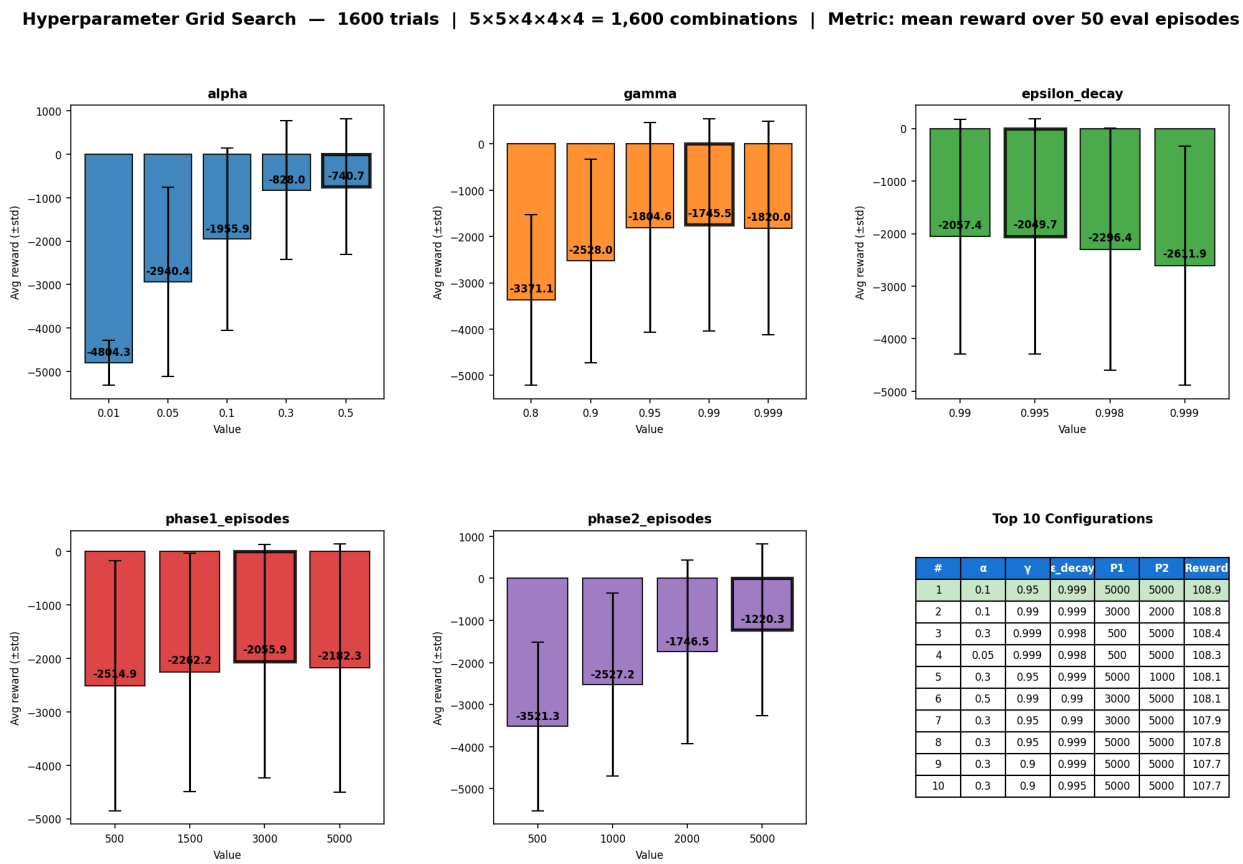


Metric	Trained	Random
Success rate	100.0%	6.8%
Mean reward	100.5	-1949.0
Std reward	15.3	385.9
Avg steps (wins)	31.0	N/A
Avg highway %	1.5%	N/A
Avg car hits/ep	0.096	N/A
Clean eps (0 hits)	90.6%	N/A
Q-table states	4757	N/A

As seen from these results, the highway usage is low, with the agent only taking highways 1.5% of the time. This, coupled with the average collision rate of 0.096, indicates the agent lost points primarily from avoiding highways instead of collisions, as a result of reduced rewards for low speed. This demonstrates that, under the proposed reward structure, the agent prioritizes safety rather than efficiency. It is also worth pointing out the improvement in the initial agent compared to the random agent. The random agent only completed the relatively small maze 6.8% of the time, with a negative mean reward. This indicates that the initial model is already performing well before hyperparameter optimization.

Grid Search Results

The grid search ran over the 1,600 designated hyperparameters computed and found which ones performed optimally. The following is a figure demonstrating how each hyperparameter influenced mean reward:



Top 10 Configurations

#	α	γ	ε_decay	P1	P2	Reward
1	0.1	0.95	0.999	5000	5000	108.9
2	0.1	0.99	0.999	3000	2000	108.8
3	0.3	0.999	0.998	500	5000	108.4
4	0.05	0.999	0.998	500	5000	108.3
5	0.3	0.95	0.999	5000	1000	108.1
6	0.5	0.99	0.99	3000	5000	108.1
7	0.3	0.95	0.99	3000	5000	107.9
8	0.3	0.95	0.999	5000	5000	107.8
9	0.3	0.9	0.999	5000	5000	107.7
10	0.3	0.9	0.995	5000	5000	107.7

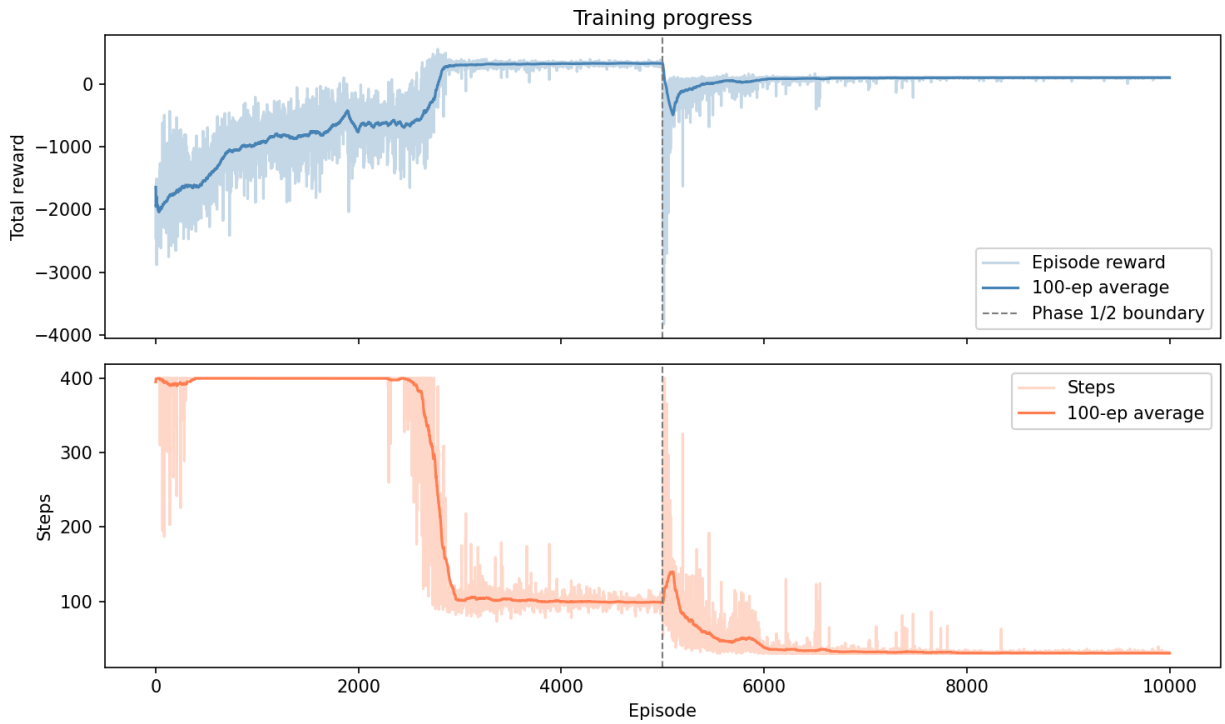
From these charts, the impact of the hyperparameter optimization is evident. Starting from the top left, the learning rate alpha has a wide distribution of effect on mean rewards. In particular, lower alpha levels tend to have a larger distribution of mean rewards. This also means that larger values tend to be more consistent, as seen with the smaller distribution for values 0.3 and 0.5. The second hyperparameter, the discount factor gamma, has narrower distributions than alpha. The general trend of larger values having narrower distributions holds true, however, as seen with a value of 0.999 compared to 0.8. Epsilon decay is similar for all values tested. No single value appears to strictly outperform the rest. Conventionally, more episodes tend to result in better performance as the model is able to train longer, however 3000 episodes in Phase 1 seems to be marginally better. This may be due to the training plateauing at around 2000. This

can also be seen in the training curves of the initial agent. Finally, the parameter for Phase 2 episodes has a clear best value: 5000. The mean and distribution for this value are more optimal than the others, signifying that training longer in a randomized world is important to agent effectiveness.

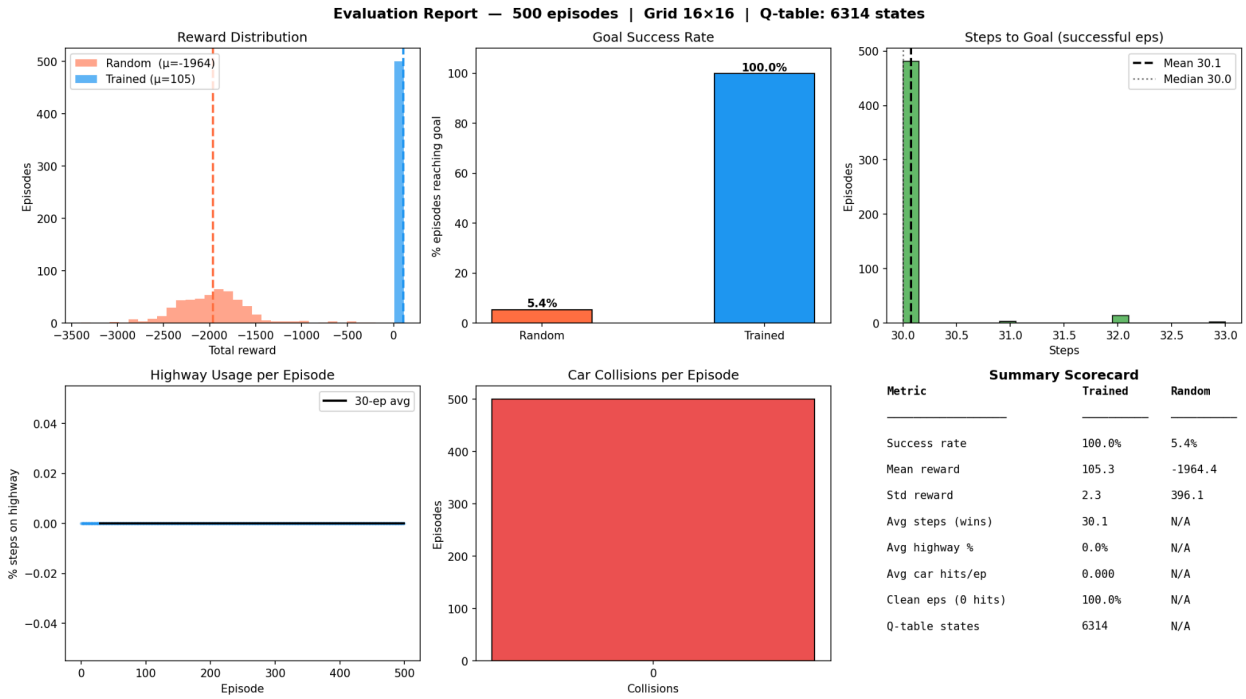
The last chart shows the top ten best performing hyperparameter combinations. It's interesting to see that the best combination is not simply all the best individual values, but instead it is a combination of all the values. The following hyperparameters will be used to train the best agent: alpha = 0.1, gamma = 0.95, epsilon_decay = 0.999, phase1_episodes = 5000, and phase2_episodes = 5000.

Results of Best Agent

The agent trained with the optimal hyperparameter values had a significantly better performance than the previous. This agent had a final mean reward of 104, almost double the initial agent, and a final mean steps count of 30 compared to the initial agent's 35.



This training curve shows a similar story as the previous agent, the agent learns during phase 1 and improves then during phase two it struggles initially but ends up doing better in the end. This agent plateaus slightly later, at around episode 3000, but seems to stay more consistent. The evaluation for this agent also reveals interesting information about the agent.



The evaluation of the best agent shows a prominent feature of this agent, which is that this agent is consistent, as seen in the mean rewards almost every run's reward was on the mean. Additionally, the agent learned to avoid highways. This may be due to the agent preferring to avoid potential collisions and take the slight penalty of taking roads. This strategy is reinforced by the fact that in evaluation there are no collisions.

Overall Results

It is clear to see the impact of the grid search on the performance of the model: mean rewards doubled and step count decreased. This improvement also reveals more information about the agents and what they learn. The best agent learned to avoid highways altogether, and brought the number of collisions to 0. This shows that the best agent prioritizes avoiding future potential collisions rather than immediate rewards. The best agent takes roads only to avoid the risk of taking a high penalty for colliding with another vehicle and is fine with taking the smaller

penalty for taking the slower route. The best agent also had a lower mean step count indicates that the initial agent also struggled with the highways, as each movement inside the highway counts as a step, this could also be another reason why highway usage is phased out as the agent sees it could get stuck there and take a lot of penalties.

Conclusions

Strengths

Three main strengths are prevalent in the final agent. First, the best agent is extremely consistent, it archives a 100% completion rate and a 99%+ collision free episode rate on a 16 x 16 grid. This is a much more consistent performance than the random agent, which had a completion rate of ~5%. Additionally, the low variance in mean reward, standard deviation of only 2.3 shows that the final policy learned is consistent and reproducible. Second, part of the success for this agent comes from the two phase training pipeline. This strategy proved to be critical and allowed for the agent to generalize much more effectively compared to only one type of training. This trend is reinforced in the training curves as the rewards are greater in phase two, then they are in phase one, this shows the agent was able to generalize to the random environments. Lastly, another factor that led to the success of the agent is the validated hyperparameters. Testing various hyperparameters with grid search proved to be the single most effective strategy to improve the model.

Weaknesses

Along with the strengths of the agent there are notable weaknesses as well. First, the Q-learning table does not really scale, even in a relatively small 16 x 16 grid, with an observation space of size 7 the state space is huge, and the state table is only ~6000 states, this means that the

agent only learned on a small fraction of the state space, meaning the agent would not generalize as well on vastly different maps. Second, rewards were manually selected and tested, this works for the purposes of the project but in reality there should have been a more structured approach to choosing the reward values. Lastly, arguably the biggest weakness of this model is the static obstacles, the cars do not move at all, this hinders the realism that the project initially was trying to achieve, and reduces the problem to more of a maze like obstacle.

Alternative Approaches

As discussed earlier, Q-learning is the default model to solve reinforcement problems but it is by far the only one. Some of the other approaches discussed in the paper, and in class, include Deep Q-Networks and Proximal Policy Optimisation. Deep Q-Networks (DQNs) essentially replace the Q-table with a neural network, this solution would directly address the scalability issues mentioned above, as now instead of using previous states it would estimate what the agent should do at any given state using the neural network. Proximal Policy Optimisation (PPO), is a similar method to Q-learning but it instead optimises probability of taking a good action rather than the state values. This method is more complicated in practice but would do better with the complicated reward structure of highways. Another approach would be to forgo the reinforcement learning approach all together and use a search algorithm such as A*, since the map is fully observable and static a simple search could be effective.

Further Improvements

There are a few concrete and implementable changes that could be made to this project in order to improve on it. The first change is to replace the Q-table with a DQN, this would be relatively simple to do with the Gymnasium interface and even with just a relatively small neural

network the current policies could be improved upon. The second change could be to change the environments in the middle of training, although phase two of training already does this, changing parameters, such as size, or number of branches could teach the agent a lot more generalization strategies and improve performance. Lastly, and perhaps most importantly, would be to add dynamic obstacles, as mentioned above, the current environment does not achieve the level of realism envisioned, but adding dynamic cars as obstacles, whether it be random movement, or through an agent of its own, could add a layer of complexity and realism to the project, worth exploring.

Works Cited

[1] V. Mnih *et al.*, “Human-level Control through Deep Reinforcement Learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015, doi: https://doi.org/10.1038/nature14236.

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” *arXiv.org*, Aug. 28, 2017. https://arxiv.org/abs/1707.06347

[3] B. R. Kiran *et al.*, “Deep Reinforcement Learning for Autonomous Driving: A Survey,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 6, pp. 1–18, 2021, doi: https://doi.org/10.1109/tits.2021.3054625.